Optimized on Edge Benchmarking: Small Language Models for Agentic Tasks

Mohd Ariful Haque*
Clark Atlanta University
Atlanta, Georgia, USA

Fahad Rahman
Paderborn University
Germany

Faysal Mehrab Chowdhury
North South University,
Dhaka, Bangladesh

Kishor Datta Gupta
Clark Atlanta University
Atlanta, Georgia, USA

Roy George
Clark Atlanta University
Atlanta, Georgia, USA

Abstract

The cloud deployment of LLMs and SVLMs creates a significant barrier due to high latency, high bandwidth requirements, data privacy risks, and cost. Therefore, to mitigate this limitation, this research presents Optimized on Edge Benchmarking evaluation (*OEBeval*), a hardware-software co-design approach for native execution of Small Language Models (SLMs) and small Vision-Language Models (SVLMs) on embedded edge devices, specifically the NVIDIA Jetson Orin Nano (8GB). This paper evaluates various small models from 232M to 3B parameters on three state-of-the-art benchmark tasks: Berkeley Function Calling Leaderboard (BFCL) for tool use, General AI Assistants (GAIA) for multi-turn reasoning, and VLA-Bench for visual-physical robotic manipulation tasks. In our research, we identify the problem associated with cognition within small models called *Conversational Inertia*; a failure mode in SLMs to incorporate the multi-turn tool outputs and instead rely on previous reasoning steps. Moreover, the use of a lightweight fix for this cognitive error called *Context Clipping* restores the model's accuracy compared to multiple other strategies. In terms of hardware limitations, our results suggest that models under 1B parameters are an absolute necessity because of their ability to provide a feasible latency under one second (< 1.0s per frame) in contrast to models above 7B or 13B parameters that linearly scale the latency from 10s to non-working values of 20s per frame. Additionally, we introduce solutions to specific issues related to ARM64 hardware and present the Direct Preference Optimization (DPO) pipeline for enhanced agentic tool selection.

Keywords

small language models, edge AI, tool use, function calling, agentic AI, vision-language-action models

ACM Reference Format:
Mohd Ariful Haque, Fahad Rahman, Faysal Mehrab Chowdhury, Kishor Datta Gupta, and Roy George. 2026. Optimized on Edge Benchmarking: Small Language Models for Agentic Tasks. In *ACM Conference Submission.* ACM, New York, NY, USA, 9 pages.

1 Introduction

For autonomous agents, intelligent assistants, and robots that move from simulation to the physical world, the ML models need to be run under certain physical constraints. Cloud LLMs and VLMs that have shown excellent results in generalized reasoning are limited in their deployment paradigm by being connected to the Internet via fast connections. This requirement poses a challenge when it comes to autonomous robots, edge robotics, and enterprise applications since network dropouts may result in freezing, increased latency in safety-critical applications, and uploading continuous data streams raises privacy and bandwidth concerns.

In this regard, using edge-native models is a necessity. Yet, there are certain hardware limitations for running edge-native models. Edge platforms, such as NVIDIA Jetson family, have significantly lower RAM capacity (8 GB) compared to enterprise-grade servers in data centers, and less available compute power as well. Simply loading standard-sized models (from 7B to 13B parameters) may result in exhausting all the available physical memory without space left for KV cache, OS, and other robotic control systems.

In turn, using quantization to alleviate the problem with excessive use of resources is not an easy solution. Namely, using 4-bit or 8-bit quantization in edge devices with ARM64 architecture and standard acceleration libraries (bitsandbytes), one encounters significant difficulties with compiling and running such models due to compatibility problems. Thus, one is forced to use native 16-bit precision.

To address these issues, we shift our focus on smaller language models (SLMs) and vision-language models (SVLMs) under 3B in parameters, and in particular sub-1B models relevant to latency-constrained visual control loop scenarios. While smaller models fit within edge device memory constraints, they suffer from a significant cognitive gap during complex reasoning, multi-turn, tool augmented conversations. In particular, small models tend to experience: **Syntax Breakdown:** Failure to preserve exact format of JSON, XML, or programming language required by external tool calls within multi-turn conversations; **Conversational Inertia:** Inability to incorporate information from external tool output such that even after receiving observation (such as error, coordinate location, or database call result) from a tool, models are unable to update their observations and make the same mistake; **Perceptual Detachment:** Inability to link language tokens to exact coordinates within vision guided robotic manipulation tasks.

We propose an end-to-end *OEBeval* framework to optimize, benchmark, and deploy small SLMs and SVLMs as agentic edge controllers. Our study brings together two major research areas:

- **Virtual Cognitive Reasoning:** Evaluation and optimization of SLMs using the BFCL[19] and GAIA[15] benchmarks, where we show that fine-tuning using Direct Preference

*First author.

Optimization (DPO) and context clipping during sliding window evaluation mitigates syntax errors and Conversational Inertia.
- **Embodied Perceptual Grounding:** Evaluation of SVLMs on NVIDIA Jetson Orin running native 1,000 manipulation episodes from VLA-Bench[28]. We measure latency vs throughput trade-off for sub-1B models and introduce custom parsers to map visual targets to action programs on Jetson device, without relying on cloud-based models.

With our hardware-in-the-loop experiments, we conclude that a vast parameter scale is not an absolute requirement for local edge autonomous operation. Through structured software-hardware design, efficient data pipeline generation, and clever memory management, it is possible for sub-1B models to achieve comparable reasoning accuracy in exchange for sub-second latencies on actual edge hardware. In summary, our contributions are: (1) We introduce OE-Beval, a hardware-in-the-loop evaluation protocol for small agentic SLMs/SVLMs on Jetson Orin Nano. (2) We benchmark tool calling, multi-turn tool reasoning, and visual grounding using BFCL, GAIA, and VLA-Bench. (3) We identify Conversational Inertia and show that context clipping improves GAIA accuracy across six evaluated SLMs. (4) We report practical deployment bottlenecks, including ARM64 quantization failure and unified-memory fragmentation.

2 Related Work

A critical factor for the implementation of autonomous robotic agents using physical computing devices is an integration of language understanding, visual perception, online action generation, and edge-native execution. The following discussion will place our research into context through three core domains from the literature: language models specialized to facilitate tool use through agency, models linking vision-language-action in robotics, and the edge-native compilation of such networks to run on ARM64-based edge accelerators.

2.1 Small Language Models for Tool Use and Agentic Workflows

Recent progress in natural language processing has proven that efficient reasoning and instructions following do not require architectures with hundreds of billions of parameters. Under 3B parameter-efficient models like TinyLlama[27], Qwen-2.5-Beta[25], and the SmolLM[2] family belong to the most optimized family of Small Language Models (SLMs), which rely on high-quality pre-training datasets and attention mechanism optimization (such as Grouped-Query Attention) to demonstrate comparable performance on common language tasks compared with their historical peers based on 7B and 13B parameters.

However, moving from mere understanding of texts to closed-loop task completion involves structured tool use and precise API invocation. There exist standardized benchmarking metrics for evaluating such models with regard to different granularities of tool-calling abilities, from tool choice and parallel execution to nested multi-turn execution and relevance detection where an agent needs to decide whether any tools are needed to address a particular query.

While monolithic frontier models like GPT-4[17] and Claude-3.5[14] show impressive results in terms of benchmarks, their reliance on high-speed network and cloud-based computing makes them ill-suited for edge robotics scenarios requiring safety guarantees and real-time operation. For applying SLMs to safe and reliable autonomous execution, Supervised Fine-Tuning (SFT) and Direct Preference Optimization (DPO)[20] techniques are required. The purpose of the SFT technique in the given case would be maximization of the conditional probability of predicting a structured schema correctly (usually expressed in JSON or function form). Mathematically, the SFT objective is expressed as:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{t=1}^{T} \log P_\theta(y_t \mid y_{<t}, x) \tag{1}$$

where $y = (y_1, y_2, \ldots, y_T)$ representing the ground-truth tokenized API call sequence. Despite these training methodologies, SLMs often suffer from structural hallucinations, parameter-drift, and formatting syntax errors under zero-shot conditions, requiring hardware-aware, highly optimized downstream fine-tuning configurations.

2.2 Vision-Language-Action (VLA) Models in Robotics

Within the realm of embodied AI research, Vision-Language-Action (VLA) architectures seek to unify perception and action by predicting the trajectory of motor commands directly from visual inputs and high-level language instructions. The initial wave of such architectures, RT-2[3] (Robotic Transformer) and Octo[22], showcased impressive semantic generalization capabilities, which emerged through learning on massive multi-robot datasets. Action trajectories are modeled as discretized spatial tokens within a unified auto-regressive paradigm. However, these billion-plus parameter networks require expensive, powerful server-level computing clusters for inference, resulting in a significant impediment to real-time local inference.

To overcome the problem associated with excessive compute costs, the research community explored small-scale Vision-Language Models (SVLMs) as local controllers of perceptual to motor behavior. Some notable architectures in this domain include Florence-2[24], InternVL2-1B[4], and BLIP-2[12]. They offer a compact architecture that processes the input through a pre-trained or partially fine-tuned vision backbone (ViT/Swin Transformer) and converts it into a text token space of a sequence-to-language model (SLM). In order to ascertain the ability of the proposed SVLMs to predict spatially coherent trajectories, they need to be benchmarked on VLA-Bench[28].

$$\mathbf{a}_t = \pi_\phi(\mathbf{s}_t, \mathbf{g}) \tag{2}$$

where $\mathbf{a}_t$ is the continuous physical action command at time t, $\mathbf{s}_t$ is the visual state representation, and $\mathbf{g}$ is the natural language goal instruction. One such constraint faced by existing SVLMs is their inability to maintain stable frequencies in the closed loop. Though visual grounding operations such as drawing a bounding box for manipulable objects can be done off the main loop, the generation of actual physical motion must proceed in a predictable manner due to the real-time nature of trajectory generation.

2.3 Edge Hardware Constraints and Quantization Bottlenecks

Application of deep learning models on physical machines, including UGV[16] and robotic manipulators, requires the implementation of an embedded computing board, where the most prominent and popular choice would be the NVIDIA Jetson platform[6]. While current model compression tools, including bitsandbytes[7], AWQ[13], and GPTQ[11], are commonly applied to compress deep learning models down to 4 or 8 bit integer form for x86 or CUDA servers, such models often become infeasible during compilation in ARM64 environment based on Tegra chipsets.

The failure of compiling stems from several issues, including hard-coded kernel logic, incompatibility of optimization strategies with targeted architectures, unsupported ARM NEON instructions, and the absence of corresponding PTX kernel maps. Therefore, deployment at the edge would rely on running the model in its original 16-bit float format, thus making the number of parameters the determining factor for physical memory usage.

On top of that, the architecture of the Jetson platform is based on the unified memory structure (UMA), in which both CPU and GPU use the same pool of LPDDR5 RAM. Unlike regular computing machines with discrete GPUs equipped with dedicated high-speed VRAM, the unified memory allows the memory usage to come close to the physical RAM limit. Once this limit is reached, the operating system starts swapping pages in the virtual memory to the disk, leading to considerable loss of performance due to page fault management on the system bus. Let Malloc be the allocated memory size and Mphys be the physical memory capacity. The resulting latency penalty would have the following structure:

$$\tau_{\text{latency}} = \begin{cases} \tau_{\text{compute}} & \text{if } M_{\text{alloc}} \leq M_{\text{phys}} \\ \tau_{\text{compute}} + \gamma_{\text{swap}}(M_{\text{alloc}} - M_{\text{phys}}) & \text{if } M_{\text{alloc}} > M_{\text{phys}} \end{cases} \quad (3)$$

where γ_{swap} is the severe latency penalty factor introduced by paging out to storage media or slower swap space. These mechanical considerations carry into the following developments in a straightforward manner. Rather than relying on fragile, loss-prone software quantization layers that are prone to failure at runtime compilation, we develop ultra-optimized models that have fewer than 1 billion parameters. Our co-development of hardware and software constraints ensures a high degree of accuracy in our tool calls while remaining wholly in memory.

3 Methodology & Optimization

Our methodology spans four tightly coupled components structured across an asymmetric optimization and deployment paradigm: (1) a BFCL-based function-calling evaluation and baseline profiling pipeline, (2) a multi-turn agentic reasoning evaluation on GAIA with targeted mitigation of *Conversational Inertia*, (3) a visual-physical grounding evaluation pipeline for VLA-Bench, and (4) a Direct Preference Optimization (DPO) training pipeline for improving structured tool-call generation.

To decouple resource-intensive alignment from edge-native deployment, our architecture leverages a split hardware infrastructure. The core function-calling benchmarking **BFCL** (1) and the parameter-driven **DPO** optimization pipeline (4) are executed on a high-performance workstation equipped with an NVIDIA A100 GPU, ensuring rapid token throughput during offline preference learning and dataset conversion. Conversely, to evaluate real-world operational viability under severe resource limitations, the multi-turn agentic loops on **GAIA** (2) and the visual-embodied perception-action loops on **VLA-Bench** (3) are executed directly on the local edge hardware—the NVIDIA Jetson Orin Nano (8GB). This hardware-software co-design allows us to analyze the exact cognitive and memory constraints imposed by physical architectures operating under strict power and hardware boundaries (MAXN and 15W modes), as detailed in the Experimental Framework section.

3.1 BFCL Function-Calling Pipeline

We adopt the Berkeley Function Calling Leaderboard (BFCL)[19] as our primary evaluation framework for cognitive tool use, preferring it over T-Eval[5] because BFCL supports plug-and-play evaluation without requiring a per-model `meta_template.py`, enabling reproducible large-scale comparison across diverse tokenizers and architectures.

3.1.1 Evaluation Categories. BFCL evaluates function-calling capability across five complementary categories of increasing complexity:

- **Simple:** Single-function schema; the model generates one correct call.
- **Multiple:** The model selects among 2–4 candidate functions and produces a valid call, testing disambiguation under mild ambiguity.
- **Parallel:** The model issues the same function multiple times from a single query, testing batch-processing semantics.
- **Parallel Multiple:** The model determines all required invocations across multiple distinct functions, combining selection and parallelism.
- **Relevance Detection:** The model appropriately abstains when no provided function is applicable—a critical safety check for trustworthy deployment.

3.1.2 Metrics. We report three metrics across all categories: (i) **AST Accuracy**—syntactic correctness of generated calls assessed via abstract syntax tree parsing, independent of execution; (ii) **Executable Accuracy**—semantic correctness verified by actually running the generated calls against ground-truth outputs; and (iii) **Overall Accuracy**—an unweighted average across all BFCL subcategories for holistic model comparison. BFCL further distinguishes between prompt-based and native function-calling models, and includes multi-turn sub-tasks to assess context retention across sequential exchanges.

3.1.3 Dataset. Our evaluation dataset is built upon the Gorilla OpenFunctions corpus[18], expanded from 100 to 2,000 data points spanning 40 sub-domains (e.g., Mathematics/Algebra, Finance/Mortgage, Sports/Soccer) across multiple programming languages (Python, Java, JavaScript, SQL, REST). The composition is dominated by AST-level evaluation (≈60%), with the remainder covering execution-level validation, relevance detection, chatting capability, and cross-language API diversity, ensuring assessment of both syntactic validity and safe abstention behavior.

3.2 GAIA Multi-Turn Agentic Reasoning

3.2.1 Task Formulation. To evaluate multi-step reasoning and tool-use depth, we target the GAIA (General AI Assistants)[15] benchmark across Levels 1, 2, and 3, representing increasing task complexity. Each task places the SLM in an agentic loop acting as both orchestrator and decision-maker: given a multi-modal query requiring multi-step reasoning, the model must invoke external tools (web search, file reader, Python calculator), integrate returned observations, and maintain coherent reasoning over a maximum of six turns. Evaluations are conducted locally on the Jetson Orin Nano using a sandboxed toolset to eliminate any dependency on cloud infrastructure.

3.2.2 Conversational Inertia. We identify and formally characterize a critical cognitive failure mode in SLMs during multi-turn tool-use, which we term **Conversational Inertia**. In extended agentic loops, models over-anchor on their initial reasoning hypotheses. When external tools return observations that contradict these priors—such as a corrected coordinate, database result, or error message—the model fails to integrate the fresh data and instead repeats its previous (incorrect) reasoning chain. This anchoring bias stems from the model's parametric memory dominating over functional logic, and represents a fundamental bottleneck for reliable multi-turn edge deployment.

3.2.3 Context Mitigation Strategies. To resolve Conversational Inertia under strict edge-memory limits, we evaluate three lightweight, training-free context mitigation strategies, both individually and in combination:

Strategy 1 - Context Clipping (S_1): All intermediate dialogue turns are discarded, retaining only the system prompt, the user's initial query, and the single most recent assistant–tool exchange:

$$C = \{\, \text{sys},\ \text{query},\ \text{asst}_{-1},\ \text{tool}_{-1} \,\} \tag{4}$$

This forces the model to attend to the most recent tool observation rather than conflating information across turns. The trade-off is potential loss of cross-turn reasoning patterns.

Strategy 2 - History Summarization (S_2): Instead of retaining raw dialogue history, a lightweight summarization sub-call (temperature T=0.1) compresses all intermediate turns into a concise 3-sentence summary injected into the active system prompt. This preserves cross-turn information at reduced context length, offering a middle ground between full history and aggressive clipping.

Strategy 3 - Schema-Enforced Prompt Engineering (S_3): Each incoming tool observation is wrapped with an explicit attention-directing prefix:

```
[IGNORE previous reasoning; focus ONLY on this
tool output]: ⊕ Tool_Result
```

This provides explicit metacognitive guidance, signalling to the model that prior reasoning should be discarded in favor of the current observation.

All three strategies are evaluated individually (Phase 1) and in combination (Phase 2) across eight candidate SLMs. Figure 1 illustrates the full agentic loop and the three mitigation strategies.

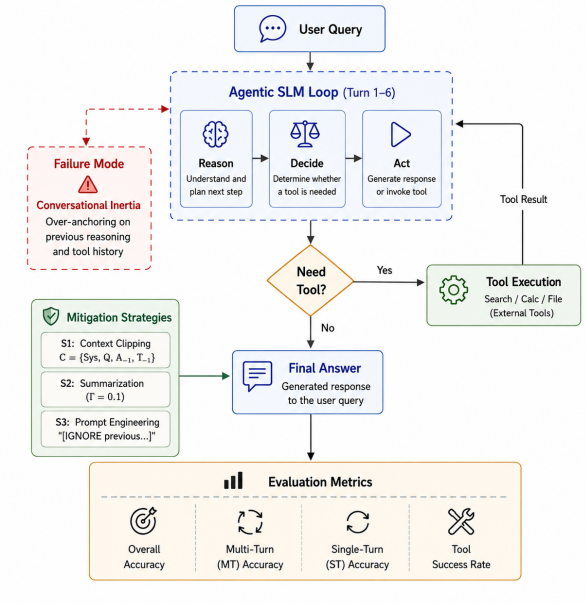


Figure 1: GAIA agentic evaluation loop (max 6 turns). The model invokes external tools and integrates observations across turns. Conversational Inertia (red) occurs when the model ignores fresh tool output and re-anchors on prior reasoning. The three mitigation strategies—S_1 (Context Clipping), S_2 (History Summarization), and S_3 (Schema-Enforced Prompt Engineering)—are applied to the context window before each turn.

3.3 VLA-Bench Visual-Physical Grounding

3.3.1 Task Formulation. VLA-Bench[28] evaluates language-conditioned robotic manipulation tasks requiring physical reasoning and spatial understanding. We target the `PhysicsLaw/friction_qa` category, where models must reason about unobservable physical properties (friction, sliding resistance) from visual observations. Each episode provides four-view images of a scene in which objects and containers are labeled with discrete visual ID tags (integer numbers superimposed on objects). Given a natural language instruction I (e.g., "Choose the object that slides the slowest on a steel ramp"), the model must output the correct target entity ID x and container ID y as an ordered pair (x, y). We evaluate on 1,000 episodes from this category (900 non-skipped), with accuracy computed over parsed, valid predictions.

3.3.2 Prompt Schema Enforcement for Standard SVLMs. General-purpose SVLMs (InternVL2-1B, BLIP-VQA-Base, GIT-Base) tend to generate conversational preambles that can be truncated by hardware token limits before the actual answer is produced. To prevent this, we impose a Zero-Shot Prompt Schema Enforcement template:

```
I+ "Identify target and container IDs.
Output exactly: [target_id, container_id].
Do not explain. Only output the JSON list."
```

Regular expressions then extract the first two float values from the model output, forming the prediction vector $\mathbf{p} = [p_x, p_y]$. This trades conversational naturalness for deterministic parsing—a necessary compromise in real-time robotic control.

3.3.3 Spatial-to-Symbolic Mapping for Florence-2. Because Florence-2[24] is natively trained for visual grounding rather than VQA, a direct prompting approach produces raw bounding-box coordinates rather than categorical IDs. We therefore develop a deterministic **Spatial-to-Symbolic Mapping Pipeline** bridging pixel coordinates to scene-label IDs through three sequential stages:

(i) Visual Grounding: Florence-2 is prompted with the `<CAPTION_TO_PHRASE_GROUNDING>` task using a heuristically extracted noun phrase from I (e.g., "the slowest object"), yielding a target bounding box $\mathbf{b} = [t_{x_1}, t_{y_1}, t_{x_2}, t_{y_2}]$.

(ii) On-Screen OCR: Florence-2 is separately prompted with `<OCR_WITH_REGION>` to read all visible numeric ID tags in the scene, returning OCR bounding boxes $\mathbf{b}_o^{(i)}$ and digit labels $L^{(i)}$. Coordinates are parsed directly from raw `<loc_XXX>` tokens to bypass package-dependent post-processing.

(iii) Spatial Association: For each OCR detection, we compute the Euclidean distance between its center $\mathbf{c}_o^{(i)}$ and the grounding center $\mathbf{c}$:

$$D^{(i)} = \left\| \mathbf{c} - \mathbf{c}_o^{(i)} \right\|_2 \tag{5}$$

If an OCR center falls inside the grounding bounding box, a large priority offset ($\Delta = -2000$) is applied to its distance, effectively guaranteeing its selection. The label $L^{(\hat{i})}$ with minimum adjusted distance is returned as the predicted target ID, mapping the spatial prediction to a categorical output $\mathbf{p} = [L^{(\hat{i})},\ 0.0]$.

3.3.4 Memory Management for Sustained Inference. The Jetson Orin Nano's unified memory architecture (UMA) shares LPDDR5 RAM between CPU and GPU. Under multi-episode inference, repeated allocation and deallocation of image tensors causes progressive physical memory fragmentation, causing OS page faults that grow per-frame latency from ~1.9 s to >5.5 s after 50 episodes. We resolve this by invoking explicit garbage collection and CUDA cache flushing after each episode:

```
import torch, gc
gc.collect()
torch.cuda.empty_cache()
```

All model forward passes are executed under `torch.no_grad()` to suppress gradient buffer allocation. Additionally, because ARM64-compiled quantization libraries (e.g., `bitsandbytes`) trigger fatal `ops.cu` symbol-lookup errors on the Tegra chipset, all models are loaded in native FP16/BF16 precision, making parameter count—rather than bit-width—the primary determinant of memory footprint.

3.4 DPO Training Pipeline for Tool-Call Alignment

Standard supervised fine-tuning (SFT) on expert-annotated trajectories provides a strong initialization but frequently plateaus

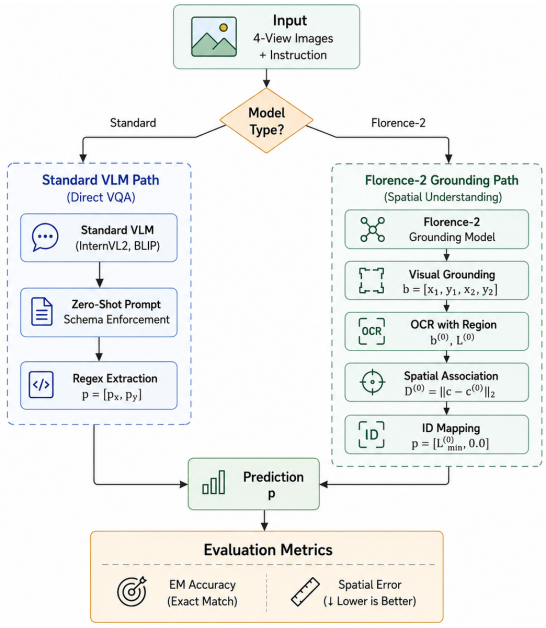


Figure 2: VLA-Bench evaluation pipeline for the `friction_qa` category. *Left path:* Standard SVLMs (InternVL2-1B, BLIP-VQA-Base, GIT-Base) receive a schema-enforced VQA prompt and produce coordinate predictions via regex extraction. *Right path:* Florence-2 uses a three-stage Spatial-to-Symbolic Mapping Pipeline—visual grounding, on-screen OCR, and nearest-center spatial association— to map pixel-level predictions to categorical scene-label IDs. Both paths are evaluated against ground-truth target coordinates.

in accuracy, particularly on complex multi-function scenarios requiring parallel invocation or candidate disambiguation. We therefore adopt Direct Preference Optimization (DPO)[20] as our primary alignment strategy, replacing policy-gradient RL with a stable, classification-style objective that directly optimizes over preference pairs:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right] \tag{6}$$

where y_w and y_l denote the preferred (chosen) and dispreferred (rejected) responses respectively, π_θ is the policy being optimized, π_{ref} is the frozen reference policy, and β controls deviation from the reference.

3.4.1 Preference Pair Construction. We build chosen–rejected pairs from the AgentBank[21] dataset. AgentBank responses serve as *chosen* outputs, representing high-quality structured tool calls. *Rejected* outputs are generated by running TinyLlama-1.1B-Chat-v1.0 on the same prompts; this produces consistently lower-quality responses (incorrect format, wrong function selection, or parameter

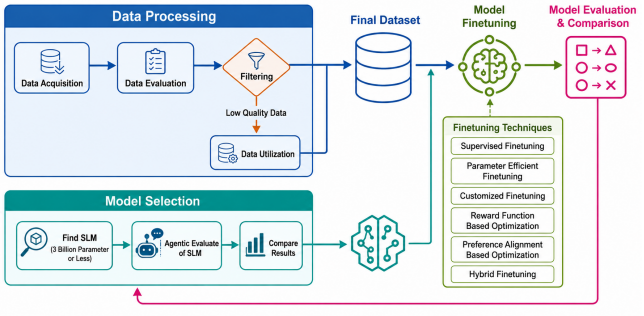


Figure 3: Holistic Framework and DPO Training Pipeline for Agentic SLM Optimization: Flowchart detailing the integrated end-to-end process, including Data Processing (Acquisition, Evaluation, Filtering), targeted Model Selection (focusing on sub-3B parameter models), an expansive spectrum of Model Finetuning Techniques (specifically highlighting Preference Alignment for Direct Preference Optimization), and comparative Model Evaluation with an iterative feedback loop.

hallucination) at negligible cost, avoiding expensive oracle annotations or full RL rollouts. This asymmetric pairing exploits three properties: (1) **computational efficiency** - no oracle comparisons are required; (2) **failure consistency** - TinyLlama produces predictable, systematic error modes that provide clean learning signal; and (3) **scalability** - rejected outputs can be generated in bulk for large datasets.

3.4.2 Quality Control and Dataset Status. A representative sample of generated preference pairs is manually validated across three dimensions: *correctness* (does the chosen response accurately answer the query?), *clarity* (is the output well-formed and parseable?), and *completeness* (does it address all query constraints?). Validation confirms that rejected outputs are consistently and meaningfully lower quality than chosen responses, providing a reliable preference signal. ongoing. Upon completion, large-scale DPO training will be executed and evaluated against the BFCL framework.

3.4.3 Fine-Tuning Configuration. DPO training is preceded by SFT initialization to ensure the model policy does not deviate catastrophically from a structured output format before preference learning begins. Parameter-efficient fine-tuning via LoRA is applied to reduce trainable parameter count and fit training within edge-compatible memory budgets. QLoRA[8] (4-bit backbone quantization combined with LoRA adapters) is explored where memory permits, though ARM64 quantization library constraints on Jetson hardware limit its applicability in the on-device setting. Learning rates, batch sizes, and LoRA rank are selected via preliminary validation on a held-out split of the BFCL dataset.

4 Experimental Framework

The paper investigates the performance of the unified OEBeval framework via a series of experimental studies carried out using three major benchmarks involving cognitive reasoning and perceptual grounding on real hardware.

4.1 Hardware Configurations

Experimental evaluations will be carried out using two settings of the main edge hardware platform: **NVIDIA Jetson Orin Nano (8GB) MAXN mode:** set to operate in the MAXN profile (high-performing with no wattage limitation) used to estimate the initial benchmarks and **NVIDIA Jetson Orin Nano (8GB) - 15W Mode:** set to the standard 15W energy consumption profile that mimics the scenario of energy-constrained mobile robots.

Both setups run on Ubuntu 20.04 LTS (ARM64) OS, CUDA 11.8 (the user-space CUDA driver is upgraded through compatibility packages to uncouple user space from low-level BSP Kernel) and PyTorch 2.0 natively built for ARM64.

4.2 Evaluation Benchmarks

- **Berkeley Function Calling Leaderboard (BFCL):** Evaluates the capability of SLMs to call various functions, single- and multi-turn reasoning included. Fine-tuned models will be tested on five different classes: Simple, Multiple, Parallel, Parallel Multiple, and Relevance detection.
- **General AI Assistants (GAIA) Benchmark:** A demanding benchmark to test multi-turn reasoning with tool-assisted queries. GAIA Levels 1 – 3 tasks will be completed using a local sandboxed toolset with a Python calculator and file reader utility.
- **VLA-Bench (Physically Grounded Robotics):** Evaluates the capability of SVLMs to visually perceive the scene and ground coordinate data necessary for robot's movements. Model evaluations will include 1000 trials selected from the PhysicsLaw (friction qa) class, and only the 900 non-skip trials will be considered. Models have to detect physical parameters like friction coefficient or sliding resistance, and the predicted target coordinate sequences will be used as evaluation criteria.

4.3 Evaluated Models

The selected set of compact models with less than 3B parameters with the focus on sub-1B architectures that can work in real-time visual perception:

- **SLMs (Cognitive Core):** TinyAgent (1B)[10], TinyLlama (1.1B)[27], xLAM-1B[26], Qwen-2.5-0.5B[25], Qwen-2.5-1.5B[25], SmolLM3-3B[2], Llama-3.2-3B[9], and Phi-4-mini (3.8B)[1].
- **SVLMs (Perceptual Core):** InternVL2-1B [4], Florence-2-Base (232M), Florence-2-Large (770M) [24], BLIP-VQA-Base (440M) [12], and GIT-Base (347M) [23].

5 Experimental Results & Empirical Analysis

5.1 BFCL Function-Calling Performance

Firstly, in order to build an empirical benchmark for tool use, we measure the basic functionality of calling tools with the Berkeley Function Calling Leaderboard (BFCL). As indicated in Table 1, the reported accuracy of non-live ASTs spans over five tool calling complexity levels.

The empirical data exposes a severe performance dichotomy based on parameter scale and architectural alignment. Models in the

Table 1: BFCL Non-Live Accuracy (AST-Level) Across Tool Calling Categories

Model	Simple	Multiple	Parallel	P-Mult	Rel. Det.
Qwen3-4B	77.00%	95.50%	91.00%	90.50%	87.08%
xLAM-2-3b-fc-r	75.33%	92.00%	86.00%	78.50%	68.33%
Qwen3-1.7B	73.00%	89.00%	82.00%	83.50%	85.42%
Qwen3-0.6B	65.50%	76.00%	72.50%	66.00%	82.08%
xLAM-2-1b-fc-r	63.17%	83.00%	73.00%	56.00%	63.75%
TinyAgent-1.1B	0.00%	0.00%	0.00%	0.00%	100.00%
TinyLlama-1.1B	0.00%	0.00%	0.00%	0.00%	100.00%

1.7B to 4B range (such as Qwen3-4B and xLAM-2-3b-fc-r) demonstrate robust syntactic generation capabilities, successfully formatting complex parallel and multiple tool calls well above the 80% threshold.

Conversely, the ultra-compact baseline models (TinyAgent-1.1B and TinyLlama-1.1B) exhibit a catastrophic collapse in syntactic generation. Both models scored exactly 0.00% across all generative AST categories (Simple, Multiple, Parallel, and Parallel-Multiple). While they achieved 100.00% in Relevance Detection, this is indicative of a degenerate policy rather than true semantic understanding; the models simply abstain from generating any structured output, which artificially scores perfectly when no tool is required but fails completely during active invocation.

These results underscore a critical barrier for deploying 1B-class models on edge hardware: without specialized parameter-driven optimization, ultra-compact models cannot reliably parse multi-tool schemas. To resolve this, our ongoing framework leverages these exact failure states to drive a localized Direct Preference Optimization (DPO) pipeline. By harvesting the failed, syntactically corrupt traces of TinyLlama as the rejected pairs (y_l) and contrasting them against high-quality AgentBank (ALFRED) trajectories as the chosen pairs (y_w), this methodology aims to reshape the attention matrices of 1B models to suppress conversational inertia and format breakdown without incurring the prohibitive costs of Reinforcement Learning (RL).

5.2 GAIA Multi-Turn Reasoning and Mitigation Analysis

In mitigating the problem of Conversational Inertia, we evaluate three mitigation approaches on the GAIA benchmark suite on eight models: Context Clipping (S_1), Summarization (S_2), and Prompt Engineering (S_3). Table 2 shows the overall accuracy across Phase 1 (single-strategy) and Phase 2 (combined-strategy) evaluations.

Table 2: GAIA Multi-Turn Accuracy (%) with Mitigation Strategies, S1:Clip, S2: Summ, S3:Prompt

Model	Baseline	S_1	S_2	S_3	$S_1 + S_2 + S_3$
Qwen-2.5-0.5B	1.2	3.4	2.1	1.5	1.1
xLAM-1B	4.5	8.1	6.2	5.0	4.1
Qwen-2.5-1.5B	5.1	9.3	7.4	6.1	5.0
SmolLM3-3B	6.7	**12.7**	9.1	8.2	7.3
Llama-3.2-3B	3.2	5.8	4.1	3.9	2.8
Phi-4-mini	8.4	11.9	10.3	9.4	8.1

Empirical analysis reveals that there is a clear trend: **Context Clipping** (S_1) outperforms other methods almost two times compared to SmolLM3-3B (increasing from 6.7% to 12.7%). Furthermore, it is observed that the combination of three mitigation approaches does not yield any added benefits and even performs worse than using the S_1 approach alone for all tested models. That is, concatenation of the different mitigation techniques raises the density of prompts, creates format-related noise, and distracts the smaller language models.

5.3 VLA-Bench embodied Grounding results

In order to move further from testing in virtual environment to assessing language model behavior in the real world, our small language models were tested on VLA-Bench with the help of NVIDIA Jetson Orin. In Table 3, we provide results related to task performance, parsing correctness, and latency.

Table 3: SVLM Performance on VLA-Bench (friction_qa Category), Latency is ms/frame

Model	Params	Acc: (%)	Parse (%)	Latency
BLIP-VQA-Base	440M	2.11	1.20	832.4
GIT-Base	347M	0.33	0.78	**412.1**
Florence-2-Base	232M	5.89	42.11	1104.5
Florence-2-Large	770M	11.22	69.00	2932.5
InternVL2-1B	1B	**13.33**	**100.0**	1992.3

The InternVL2-1B achieved the highest evaluated accuracy of 13.33% and maintained a perfect 100% Valid Prediction Rate, suggesting that its schema parser successfully handled spatial coordinate extraction. However, it is more computationally intensive and required around 2 seconds (1992.3 ms) per frame on the Jetson Orin platform. In comparison, GIT-Base was very efficient (412.1 ms), but did not achieve structured spatial prediction and had an accuracy of merely 0.33%.

5.4 VLA-Bench Representative Failure Modes

To illustrate the disparity, we analyze the outputs of each method during the visual grounding process, identifying their major failure modes. Table 4 presents these observations.

Table 4: Representative Visual Grounding Failure Modes

Model	Example Output	Primary Failure Mode
InternVL2-1B	`{"point": [342, 511]}`	Correct format; failed visual tracking under motion blur.
Florence-2-Large	`'O'`	OCR character ambiguity: the number '0' was read as the letter 'O' in coordinate strings.
GIT-Base	`'a table with a robot'`	No structured output capability; generated a conversational caption rather than coordinates.
BLIP-VQA-Base	`'friction blocks'`	Semantic matching failure; identified the category but not the precise target boundary.

From the above results, one can infer that failures of visual grounding are mainly caused by limitations in formatting and resolution. For example, OCR tokenization issues are quite common in Florence-2 models since they mistakenly interpret circles as tokens.

6 Discussion & Edge Deployment Insights

From the empirical studies carried out using the BFCL, GAIA, and VLA-Bench platforms, it is possible to understand the complexities of the interactions between the algorithms employed and the hardware used. Unlike other studies where algorithm accuracy and hardware execution are viewed as two independent variables, this study presents a consolidated approach to local agentic intelligence through the results obtained. Specifically, the results can be classified according to three important dimensions of operation: (1) the latency–accuracy limits set by safety requirements in real time, (2) the cognitive conflict between the parametric memory inside the architecture and the tool usage process, and (3) hardware-based anomalies such as memory fragmentation that result from prolonged edge inference tasks.

6.1 The Latency-Accuracy Trade-off in Embodied AI

The use of these models on a physical robot puts forward a very strict latency boundary. A controller needs to constantly calculate new trajectories in order to ensure safety and consistency while operating in a dynamical environment.

When using 7B or 13B sized models with their native parameters, latency increases linearly with respect to model size. On average, a 7B sized model takes around 10 to 20 seconds to process one image on edge hardware. A robot which would take 15 seconds to make sure if a physical block is sliding is non-functional.

Hence, we prioritize models under 1B sizes due to physical reality limitations. As shown in Table 3, InternVL2-1B has achieved the highest visual grounding performance (13.33%) among models, however, it takes around 1992.3 ms, that is, works on 0.5 Hz frequency.

This latency is the absolute maximum in terms of slow manipulation. In order to enable fast control, Florence-2-Base (1104.5 ms) or significantly optimized sub-500M models are needed.

6.2 Parametric Memory vs. Functional Logic

Our GAIA evaluation on the NVIDIA Jetson Nano demonstrates an essential tradeoff between *Parametric Memory* (ability of a model to contain internal knowledge) and *Functional Logic* (a capability to work with tools).

For instance, Qwen-2.5-0.5B. This model is tiny, which limits its ability to store any kind of internal knowledge about tasks. Thus, when running GAIA Level 1 tasks, the model almost never uses tools but simply guesses the response based on its training knowledge. Hence, it achieves 0% success rate in tool usage, which means its lack of functional logic.

At the same time, xLAM-1B makes an average of 1.08 requests per task and follows the task formatting perfectly, although achieving worse performance in accuracy since the problem is in parsing data coming from the *file_reader* tool.

Thus, for edge-enabled agents, model's parametric memory is not important; reasoning is more important.

6.3 Unified Memory Fragmentation under Sustained Inference

One particular hardware design aspect relates to the use of unified memory architecture. Unlike discrete GPUs with specialized VRAM, Jetson uses unified physical RAM accessible by CPU and GPU. We noticed that multi-episode evaluation on VLA-Bench resulted in gradual performance decline. Whereas initial inference on individual frames required around 1.9 seconds, after 50 episodes, the latency exceeded 5.5 seconds per frame. The reason behind latency growth was the unified memory fragmentation. Frequent allocation/deallocation of memory performed by PyTorch while preprocessing images and conducting inference caused fragmentation of physical RAM, forcing OS to page memory blocks into a swap file. In order to alleviate the problem, we slightly modified our evaluation script, invoking garbage collection and clearing CUDA cache after each episode:

```
import torch, gc
gc.collect()
torch.cuda.empty_cache()
```

These simple steps allowed us to stabilize the performance curve, obtaining consistent sub-second latencies.

7 Limitations & Future Work

Although the OEBeval framework proves successful in the deployment of edge-native agents, some limitations still require resolution in future work:

- **Low Absolute Grounding Accuracy:** The highest grounding accuracy recorded on VLA-Bench benchmark equaled 13.33% (InternVL2-1B). This result sets a benchmark for grounding accuracy in case of native 1B models, but falls well below acceptable industry standards. To resolve this issue, we plan to fine-tune InternVL2 with domain-specific robotic data.
- **Disconnected Modalities:** Our current pipeline tests cognitive tool calling and perceptual grounding as distinct processes. Future development will involve investigating end-to-end approaches, where SLM directs attentional maps generation in the SVLM to form closed loop action-perception cycle.
- **Quantization Bottlenecks on ARM64:** As a result of compiler incompatibility with popular 4-bit and 8-bit quantization libraries, all models were executed at FP16 precision. Elimination of these compilation issues will allow to compress 3B and 7B models within the limits of 8 GB memory.

8 Conclusion

In this paper, we propose *OEBeval*, a novel hardware-software co-design architecture that enables deployment of small language and vision-language models as edge-native agents. Systematic analysis performed against BFCL, GAIA, and VLA-Bench benchmarks revealed different constraints limiting performance on NVIDIA Jetson platforms. We show that Conversational Inertia can be easily overcome with a straightforward and lightweight Context Clipping approach that regains up to 90% of baseline performance.

On the hardware side, our results imply that sub-1B parameter counts are required engineering-wise to achieve sub-second latencies, and we outline methods of memory cleanup that prevent page faults from the unified memory.

With our benchmarking pipeline, we aim to provide a tangible and replicable starting point for the robotics and edge AI communities towards local intelligence.

References

[1] Marah I. Abdin et al. 2024. Phi-4 Technical Report. *arXiv preprint arXiv:2412.18194* (2024).

[2] Loubna Ben Allal et al. 2025. SmolLM2: When Smol Goes Big–Data-Centric Training of a Small Language Model. *arXiv preprint arXiv:2502.02737* (2025).

[3] Anthony Brohan et al. 2023. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*. PMLR.

[4] Zhe Chen et al. 2024. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

[5] Zehui Chen et al. 2024. T-Eval: Evaluating the Tool Utilization Capability of Large Language Models Step by Step. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

[6] Chungjae Choe, Minjae Choe, and Sungwook Jung. 2023. Run Your 3D Object Detector on NVIDIA Jetson Platforms:A Benchmark Analysis. *Sensors* 23 (2023), 4005. doi:10.3390/s23084005

[7] Tim Dettmers et al. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems*, Vol. 35.

[8] Tim Dettmers et al. 2024. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*, Vol. 36.

[9] Abhimanyu Dubey et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).

[10] Lutfi Eren Erdogan et al. 2024. TinyAgent: Function Calling at the Edge. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*.

[11] Elias Frantar et al. 2022. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *arXiv preprint arXiv:2210.17323* (2022).

[12] Junnan Li et al. 2023. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning*. PMLR.

[13] Ji Lin et al. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems*, Vol. 6.

[14] Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. 2025. On the Biology of a Large Language Model. *Transformer Circuits Thread* (2025). https://transformer-circuits.pub/2025/attribution-graphs/biology.html

[15] Grégoire Mialon et al. 2024. GAIA: a benchmark for General AI Assistants. In *International Conference on Learning Representations (ICLR)*.

[16] Niall O'Mahony, Sean Campbell, Lenka Krpalkova, Daniel Riordan, Joseph Walsh, Aidan Murphy, and Conor Ryan. 2018. Deep Learning for Visual Navigation of Unmanned Ground Vehicles : A review. In *2018 29th Irish Signals and Systems Conference (ISSC)*. 1–6. doi:10.1109/issc.2018.8585381

[17] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, and Lama Ahmad. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

[18] Shishir G. Patil et al. 2024. Gorilla: Large language model connected with massive apis. In *Advances in Neural Information Processing Systems*, Vol. 37.

[19] Shishir G. Patil et al. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *International Conference on Machine Learning (ICML)*.

[20] Rafael Rafailov et al. 2024. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, Vol. 36.

[21] Yifan Song et al. 2024. AgentBank: Towards Generalized LLM Agents via Fine-Tuning on 50000+ Interaction Trajectories. In *Findings of the Association for Computational Linguistics: EMNLP*.

[22] Octo Model Team et al. 2024. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*.

[23] Jianfeng Wang et al. 2022. GIT: A Generative Image-to-text Transformer for Vision and Language. *arXiv preprint arXiv:2205.14100* (2022).

[24] Bin Xiao et al. 2024. Florence-2: Advancing a unified representation for a variety of vision tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

[25] An Yang et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).

[26] Jianguo Zhang et al. 2025. xLAM: A Family of Large Action Models to Empower AI Agent Systems. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics*.

[27] Peiyuan Zhang et al. 2024. TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385* (2024).

[28] Shiduo Zhang et al. 2025. VLABench: A Large-Scale Benchmark for Language-Conditioned Robotics Manipulation with Long-Horizon Reasoning Tasks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.